

Группа:	Z3342	К работе допущен	_____
Студент:	Петров М.Ю.	Работа выполнена	_____
Преподаватель:	Маслов И.Д.	Отчёт принят	_____

Рабочий протокол и отчёт по лабораторной работе № 4 [C++ & UNIX]: C++ PROCESSES / THREADS

Цель

Познакомить студента с принципами параллельных вычислений. Составить несколько программ с простейшими вычислительными действиями, чтобы освоить принципы параллельных вычислений (когда одни алгоритмы зависят / не зависят от других).

Задачи

Требуется выполнить вычисления:

- Формула 1: $f(x) = x^2 - x^2 + 4x - 5x + x + x$
- Формула 2: $f(x) = x + x$
- Формула 3: $f = f_1 + f_2 - f_1$

Решение

[C++ SEQUENCE] Последовательные вычисления

```
#include <iostream>
#include <chrono>

using namespace std;
using namespace chrono;

double f1(double x) {
    return x*x - x*x + x*4 - x*5 + x + x;
}

double f2(double x) {
    return x + x;
}

void run_sequence(int n) {
    auto start = high_resolution_clock::now();

    double r1 = 0, r2 = 0;

    for (int i = 0; i < n; i++) {
        r1 += f1(i);
    }

    for (int i = 0; i < n; i++) {
        r2 += f2(i);
    }

    double r3 = r1 + r2 - r1;

    auto end = high_resolution_clock::now();
```

```

        cout << "SEQ_" << n << "):_"
              << duration_cast<nanoseconds>(end - start).count()
              << "_ns\n";
    }

int main() {
    int n1 = 10000;
    int n2 = 100000;
    run_sequence(n1);
    run_sequence(n2);
}

```

Результат:

```

SEQ (10000): 147049 ns
SEQ (100000): 1395944 ns

```

[C++ THREADS] Параллельные вычисления через потоки

```

#include <iostream>
#include <chrono>
#include <thread>

using namespace std;
using namespace chrono;

double f1(double x) {
    return x*x - x*x + x*4 - x*5 + x + x;
}

double f2(double x) {
    return x + x;
}

void calc_f1(int n, double& res) {
    for (int i = 0; i < n; i++) res += f1(i);
}

void calc_f2(int n, double& res) {
    for (int i = 0; i < n; i++) res += f2(i);
}

void run_threads(int n) {
    double r1 = 0, r2 = 0;

    auto start = high_resolution_clock::now();

    thread t1(calc_f1, n, ref(r1));
    thread t2(calc_f2, n, ref(r2));

    t1.join();
    t2.join();

    double r3 = r1 + r2 - r1;

    auto end = high_resolution_clock::now();

    cout << "THREAD_" << n << "):_"
          << duration_cast<nanoseconds>(end - start).count()
          << "_ns\n";
}

int main() {
    int n1 = 10000;

```

```

    int n2 = 100000;
    run_threads(n1);
    run_threads(n2);
}

```

Результат

```

THREAD (10000): 647955 ns
THREAD (100000): 2759029 ns

```

[C++ PROCESS] Параллельные вычисления через процессы

```

#include <iostream>
#include <chrono>
#include <unistd.h>
#include <sys/wait.h>

using namespace std;
using namespace chrono;

double f1(double x) {
    return x*x - x*x + x*4 - x*5 + x + x;
}

double f2(double x) {
    return x + x;
}

void run_process(int n) {
    int pipefd[2];
    pipe(pipefd);

    auto start = high_resolution_clock::now();

    pid_t pid = fork();

    if (pid == 0) {
        close(pipefd[0]);
        double r2 = 0;
        for (int i = 0; i < n; i++) r2 += f2(i);
        write(pipefd[1], &r2, sizeof(r2));
        close(pipefd[1]);
        exit(0);
    } else {
        double r1 = 0;
        for (int i = 0; i < n; i++) r1 += f1(i);

        wait(NULL);

        double r2;
        read(pipefd[0], &r2, sizeof(r2));

        double r3 = r1 + r2 - r1;

        auto end = high_resolution_clock::now();

        cout << "PROC_" << n << "):_"
              << duration_cast<nanoseconds>(end - start).count()
              << "_ns\n";
    }
}

int main() {
    int n1 = 10000;

```

```
int n2 = 100000;  
run_process(n1);  
run_process(n2);  
}
```

Результат

```
PROC (10000): 1244302 ns  
PROC (100000): 1964235 ns
```

Заключение

В ходе работы были изучены:

- последовательные вычисления
- параллельные вычисления
- потоки (threads)
- процессы (fork)
- измерение времени выполнения